

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment :	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined data path structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	B. Brahmendra babu	Reg. No.:	192312439
Department:	ECE	Date:	06-08-26

Code of Conduct:

I Brahmendra babu (192312439) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Question 1

Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Answer

Aim

To design and simulate a register transfer system that performs data transfer between registers and executes basic arithmetic and logical operations using the Arithmetic Logic Unit (ALU).

Theory

A processor contains several registers that temporarily store data during program execution. Data is transferred from one register to another through register transfer operations.

The **Arithmetic Logic Unit (ALU)** performs arithmetic operations such as addition and subtraction, and logical operations such as AND and OR. During execution, the Control Unit sends control signals to transfer data between registers and instructs the ALU to perform the required operation.

Experimental Procedure

Step 1: Create four registers named **R1, R2, R3, and R4**.

Assume the following values:

R1 = 15

R2 = 10

R3 = 0

R4 = 0

Step 2: Perform Register Transfer

$R3 \leftarrow R1$

Now,

R3 = 15

This operation transfers the contents of Register R1 into Register R3.

Step 3: Perform Addition

Operation:

$$\mathbf{R4 \leftarrow R1 + R2}$$

Calculation:

$$15 + 10 = \mathbf{25}$$

Therefore,

$$\mathbf{R4 = 25}$$

The ALU performs the addition and stores the result in Register R4.

Step 4: Perform Subtraction

Operation:

$$\mathbf{R4 \leftarrow R1 - R2}$$

Calculation:

$$15 - 10 = \mathbf{5}$$

Therefore,

$$\mathbf{R4 = 5}$$

The ALU subtracts the value of R2 from R1 and stores the result in R4.

Step 5: Perform Logical AND

Binary Values:

$$15 = \mathbf{1111}$$

$$10 = \mathbf{1010}$$

AND Operation:

1111

1010

1010

$$\text{Result} = \mathbf{1010_2 = 10}$$

Therefore,

$$\mathbf{R4 = 10}$$

Step 6: Perform Logical OR

Binary Values:

15 = **1111**

10 = **1010**

OR Operation:

1111

1010

1111

Result = **1111₂ = 15**

Therefore,

R4 = 15

Observation

Operation	Result
R3 $\leftarrow$ R1	15
R4 $\leftarrow$ R1 + R2	25
R4 $\leftarrow$ R1 - R2	5
R4 $\leftarrow$ R1 AND R2	10
R4 $\leftarrow$ R1 OR R2	15

Analysis

The register transfer operations successfully moved data between registers without modifying the original values. The ALU correctly performed addition, subtraction, AND, and OR operations based on the control signals provided by the processor. This experiment demonstrates how registers and the ALU work together to execute instructions efficiently.

Result

The register transfer system was successfully simulated. Data transfer between registers was performed correctly, and the ALU executed all arithmetic and logical operations accurately.

The experiment shows that registers temporarily store data while the ALU performs computations during processor execution.

Question 2

Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

Answer

Aim

To study the working of **Single-Bus** and **Multi-Bus** organizations by performing data transfer operations between registers, memory, and I/O devices, and compare their performance and efficiency.

Theory

A **bus** is a communication path that transfers data between different parts of a computer system. There are two common bus organizations: **Single-Bus** and **Multi-Bus**. In a Single-Bus system, all components share one common bus, so only one data transfer can occur at a time. In a Multi-Bus system, multiple buses are available, allowing several data transfers to occur simultaneously. This improves the speed and overall performance of the processor.

Experimental Procedure

Single-Bus Organization

Assume the following values:

- Register R1 = **20**
- Register R2 = **15**
- Memory Location M = **100**
- I/O Device = **Input Data**

Step 1: Transfer data from Memory to Register.

R1 ← **M**

Memory sends the value **100** to Register R1.

Step 2: Transfer data from Register to I/O.

I/O ← **R1**

The value stored in Register R1 is transferred to the I/O device.

Since there is only one bus, these operations are performed **one after another**.

Multi-Bus Organization

Using multiple buses, the following operations can occur simultaneously.

- **Bus 1:** Memory → Register R1
- **Bus 2:** Register R2 → ALU
- **Bus 3:** ALU → Register R3

Since different buses are used, all three operations can be performed at the same time without waiting for each other.

Observation

Single-Bus Organization

Operation	Time Required
Memory → Register	1 Cycle
Register → I/O	1 Cycle
Total	2 Clock Cycles

Multi-Bus Organization

Operation	Time Required
Memory → Register	Parallel
Register → ALU	Parallel
ALU → Register	Parallel
Total	1 Clock Cycle

Comparison

Single-Bus	Multi-Bus
Uses one common bus.	Uses multiple buses.
Only one transfer at a time.	Multiple transfers occur simultaneously.
Higher execution time.	Lower execution time.
Simple hardware design.	More complex hardware design.
Lower performance.	Higher performance.

Analysis

The Single-Bus organization is simple but becomes slow because every component has to wait for the bus to become free. This creates a bottleneck when multiple data transfers are required.

The Multi-Bus organization allows different components to communicate at the same time. As a result, data transfer becomes faster, processor utilization increases, and the overall system performance improves.

Result

The experiment successfully demonstrated both **Single-Bus** and **Multi-Bus** organizations. It was observed that the **Multi-Bus organization** provides better performance by allowing parallel data transfer, reducing bus contention, and minimizing execution time. Therefore, Multi-Bus systems are preferred in modern computer architectures due to their higher speed and efficiency.

Question 3

Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

Answer

Aim

To simulate a **5-stage instruction pipeline** and compare its performance with a non-pipelined processor by calculating the speedup and throughput.

Theory

Instruction pipelining is a technique used to improve processor performance by dividing the execution of an instruction into different stages. Instead of executing one instruction at a time, multiple instructions are processed simultaneously, with each instruction occupying a different stage of the pipeline. A typical processor pipeline consists of five stages:

- **IF** – Instruction Fetch
- **ID** – Instruction Decode
- **EX** – Execute
- **MEM** – Memory Access
- **WB** – Write Back

This technique increases the instruction throughput and improves CPU performance.

Experimental Procedure

Consider the following instructions:

- **I1:** ADD R1, R2, R3
- **I2:** SUB R4, R5, R6
- **I3:** AND R7, R8, R9
- **I4:** OR R10, R11, R12

Pipeline Execution

Clock Cycle	IF	ID	EX	MEM	WB
1	I1	-	-	-	-
2	I2	I1	-	-	-
3	I3	I2	I1	-	-
4	I4	I3	I2	I1	-

Clock Cycle	IF	ID	EX	MEM	WB
5	-	I4	I3	I2	I1
6	-	-	I4	I3	I2
7	-	-	-	I4	I3
8	-	-	-	-	I4

Observation

Non-Pipelined Processor

Each instruction requires **5 clock cycles**.

For **4 instructions**:

Total Clock Cycles = $4 \times 5 = 20$ cycles

Pipelined Processor

For 4 instructions:

Total Clock Cycles = **8 cycles**

Performance Calculation

Speedup

= Time taken by Non-Pipelined Processor $\div$ Time taken by Pipelined Processor

= $20 / 8$

= **2.5**

Throughput

Non-Pipelined = $4/20 = 0.2$ instructions per cycle

Pipelined = $4/8 = 0.5$ instructions per cycle

Analysis

The pipelined processor executes multiple instructions simultaneously by dividing execution into five stages. This significantly reduces the total execution time and increases the number

of instructions completed per clock cycle. Compared to the non-pipelined processor, the pipelined processor achieves higher throughput and better utilization of hardware resources.

Result

The simulation of the **5-stage instruction pipeline** was successfully completed. It was observed that pipelining reduced the execution time from **20 clock cycles to 8 clock cycles**, achieving a **speedup of 2.5** and improving the instruction throughput. Therefore, pipelining is an effective technique for enhancing processor performance.

Question 4

Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

Answer

Aim

To study different types of pipeline hazards and implement suitable techniques to reduce their impact on processor performance.

Theory

Pipeline hazards occur when the normal execution of instructions is interrupted. These hazards reduce processor efficiency by introducing delays. The three main types of hazards are:

- **Data Hazard**
- **Control Hazard**
- **Structural Hazard**

These hazards can be minimized using techniques such as **stalling**, **data forwarding**, and **branch prediction**.

Experimental Procedure

1. Data Hazard

Example:

I1 : ADD R1, R2, R3

I2 : SUB R4, R1, R5

The second instruction depends on the result produced by the first instruction.

Solution: Use **Data Forwarding** to send the result directly to the next instruction without waiting for Write Back.

2. Control Hazard

Example:

BEQ R1, R2, LABEL

The processor does not know which instruction to fetch until the branch decision is made.

Solution: Use **Branch Prediction** to predict the branch direction and continue execution.

3. Structural Hazard

Example:

Both Instruction Fetch and Memory Access require the same memory at the same time.

Solution: Use separate instruction and data memories or provide additional hardware resources.

Observation

Hazard	Cause	Solution
Data Hazard	Dependency between instructions	Data Forwarding
Control Hazard	Branch instructions	Branch Prediction
Structural Hazard Resource conflict		Separate Hardware Resources

Analysis

Pipeline hazards increase the execution time by causing stalls in the pipeline. Data forwarding reduces waiting time by providing data directly to the next instruction. Branch prediction minimizes delays caused by branch instructions, while additional hardware resources eliminate structural conflicts. These techniques improve pipeline efficiency and processor performance.

Result

The different pipeline hazards were successfully identified and analyzed. By applying **data forwarding**, **branch prediction**, and **resource optimization**, the execution delays were minimized, resulting in improved pipeline performance and better processor efficiency.

Question 5

Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Answer

Aim

To study the working of a **Superscalar Processor** and analyze the performance improvements achieved through **Out-of-Order Execution**, **Speculative Execution**, **Hyper-Threading**, and **Simultaneous Multithreading (SMT)**.

Theory

A superscalar processor can execute more than one instruction in a single clock cycle. It contains multiple execution units, allowing parallel instruction execution. Modern processors further improve performance using techniques such as Out-of-Order Execution, Speculative Execution, Hyper-Threading, and Simultaneous Multithreading (SMT).

Experimental Procedure

The processor executes multiple independent instructions simultaneously.

Example:

Instruction 1 → Integer ALU

Instruction 2 → Floating Point Unit

Instruction 3 → Load/Store Unit

All three instructions are executed in the same clock cycle because separate execution units are available.

The processor also performs:

- **Out-of-Order Execution:** Executes independent instructions before waiting instructions.
- **Speculative Execution:** Executes predicted branch instructions before the actual branch result.
- **Hyper-Threading:** Allows one processor core to execute two threads simultaneously.
- **Simultaneous Multithreading (SMT):** Executes instructions from multiple threads in parallel using available processor resources.

Observation

Technique	Benefit
Superscalar Execution	Executes multiple instructions per clock cycle
Out-of-Order Execution	Reduces waiting time due to dependencies
Speculative Execution	Reduces branch delay
Hyper-Threading	Improves CPU utilization
Simultaneous Multithreading (SMT) Increases instruction throughput	

Analysis

The superscalar processor makes better use of available execution units by executing independent instructions in parallel. Out-of-order execution avoids unnecessary delays, speculative execution reduces branch penalties, and Hyper-Threading and SMT improve processor utilization by executing multiple threads simultaneously. These techniques significantly increase throughput and overall processor performance.

Result

The study successfully demonstrated that superscalar processors execute multiple instructions simultaneously, resulting in improved throughput and better hardware utilization. The use of **Out-of-Order Execution**, **Speculative Execution**, **Hyper-Threading**, and **SMT** further

enhances processor performance, making modern processors faster and more efficient than traditional single-issue processors.